

Mathematics of the GigaPose Adaptive Tracking System

Contents

1 System overview

The tracking system has two distinct parts:

1. **run_tracking**: deterministic detection association, hypothesis generation, CAD rendering, pose scoring, local refinement, and temporal tracking.
2. **train_recovery**: training of an optional small neural network that proposes pose corrections when a track is uncertain or lost.

The main tracker is not trained. It is a deterministic search and optimization procedure. Only the optional recovery head is learned. Even when the learned head is used, its corrected pose is rendered and checked by the deterministic CAD score before it can be selected.

2 Pose representation

Every object pose is represented as a camera-from-object transformation

$$T_{co} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}, \quad R \in \text{SO}(3), \quad t = [t_x, t_y, t_z]^T \in \mathbb{R}^3. \quad (1)$$

The pose transforms a point in CAD/object coordinates into camera coordinates:

$$X_c = RX_o + t. \quad (2)$$

Rotation changes are represented by an axis-angle vector $\omega \in \mathbb{R}^3$:

$$\Delta R = \text{Exp}([\omega]_{\times}), \quad (3)$$

where $[\omega]_{\times}$ is the skew-symmetric cross-product matrix. The exponential map converts a vector in the Lie algebra $\mathfrak{so}(3)$ into a valid rotation in $\text{SO}(3)$. This prevents an update from producing a non-orthogonal rotation matrix. The implementation is in `tracking/geometry.py`.

Part I

Runtime tracking

3 Multi-car association

Before refining poses, the tracker must decide which detection in the current frame belongs to each existing track. This is an assignment problem, not a pose-estimation problem.

3.1 Basic geometric cost

For track i and detection j , the basic cost is

$$C_{ij} = w_{\text{IoU}}(1 - \text{IoU}(B_i, B_j)) + w_c \min \left(\frac{\|c_i - c_j\|_2 / d_{\text{image}}}{d_{\text{max}}}, 1 \right), \quad (4)$$

where

$$d_{\text{image}} = \sqrt{W^2 + H^2}. \quad (5)$$

B_i and B_j are bounding boxes, while c_i and c_j are their centers. The default weights are

$$w_{\text{IoU}} = 0.65, \quad w_c = 0.35. \quad (6)$$

A low cost means that a detection overlaps the previous track box and has a nearby center.

3.2 The Hungarian algorithm

All pairwise costs form a matrix. For two tracks and two detections, for example,

$$C = \begin{bmatrix} C_{0,0} & C_{0,1} \\ C_{1,0} & C_{1,1} \end{bmatrix} = \begin{bmatrix} 0.2 & 0.8 \\ 0.7 & 0.1 \end{bmatrix}. \quad (7)$$

The two possible complete assignments have costs

$$(0 \rightarrow 0, 1 \rightarrow 1) : \quad 0.2 + 0.1 = 0.3, \quad (8)$$

$$(0 \rightarrow 1, 1 \rightarrow 0) : \quad 0.8 + 0.7 = 1.5. \quad (9)$$

The Hungarian algorithm selects the first assignment. More generally, it solves

$$\min_{x_{ij}} \sum_{i,j} C_{ij} x_{ij} \quad (10)$$

subject to one-to-one assignment constraints

$$x_{ij} \in \{0, 1\}, \quad \sum_j x_{ij} \leq 1, \quad \sum_i x_{ij} \leq 1. \quad (11)$$

Thus, two tracks cannot claim the same detection and one track cannot claim two detections. The algorithm finds the globally best assignment under the provided cost; it does not create identity information that is absent from that cost.

3.3 HSV appearance descriptor

When identity-aware association is enabled, the tracker computes a color histogram inside each instance mask. RGB pixels are converted to HSV:

- H : hue, such as red, blue, green, or yellow;
- S : saturation, or how strongly colored a pixel is;
- V : value, or brightness.

The current implementation uses only H and S , with 24 hue bins and 16 saturation bins. The result is a 384-dimensional normalized histogram

$$h(k) \geq 0, \quad \sum_k h(k) = 1. \quad (12)$$

Hue and saturation are used instead of raw RGB because brightness and color are more entangled in RGB. Omitting V makes the descriptor less sensitive to illumination, shadow, haze, and exposure changes. This helps when the same car moves from sunlight into shadow.

The approach also has limitations. Hue is unreliable for gray, black, and white cars; identical liveries can have almost identical histograms; the histogram discards spatial layout; and occlusion can change the observed color distribution.

3.4 Bhattacharyya appearance distance

For two normalized histograms p and q , the Bhattacharyya coefficient is

$$BC(p, q) = \sum_k \sqrt{p_k q_k}. \quad (13)$$

The corresponding distance used for normalized histograms is

$$d_B(p, q) = \sqrt{1 - BC(p, q)}. \quad (14)$$

Therefore,

$$d_B \approx 0 \implies \text{similar color distributions}, \quad (15)$$

$$d_B \approx 1 \implies \text{different color distributions}. \quad (16)$$

The stored track appearance is updated with an exponential moving average:

$$h_t = \text{normalize}(\mu h_{t-1} + (1 - \mu) h_{\text{det}}), \quad \mu = 0.85. \quad (17)$$

3.5 Identity-aware association cost

When all optional signals are available, the cost numerator is

$$\tilde{C}_{ij} = w_{\text{IoU}}(1 - \text{IoU}_{\text{box}}) + w_c d_{\text{center}} + w_{\text{mask}}(1 - \text{IoU}_{\text{mask}}) + w_{\text{app}} d_B + w_{\text{id}} E_{\text{id}}, \quad (18)$$

and it is normalized by the sum of weights for the available terms:

$$C_{ij} = \frac{\tilde{C}_{ij}}{\sum_{k \in \mathcal{A}} w_k}. \quad (19)$$

Here, $E_{\text{id}} = 0$ for matching external IDs and 1 for different external IDs. Strict external-ID mode can forbid a conflicting match rather than merely penalizing it.

Identity-aware association is enabled with

`--identity-aware-association`

Stable external instance IDs can additionally be used with

`--use-external-ids`

External IDs should not be enabled if the detector independently reassigns instance numbers in every frame.

If two cars have the same color, d_B is nearly equal for both assignments and contributes little information. The tracker then relies on box geometry, mask overlap, motion, and any stable external ID. If two identical cars fully cross or occlude each other and no persistent identity cue exists, the assignment is fundamentally ambiguous and an identity swap is possible.

The association implementation is in `tracking/association.py`.

4 Temporal pose prediction

For consecutive poses, define translation and rotation increments as

$$\Delta t_k = t_k - t_{k-1}, \quad (20)$$

$$\Delta \omega_k = \text{Log}(R_k R_{k-1}^T). \quad (21)$$

The tracker estimates velocity over a short history using the component-wise median:

$$v_t = \text{median}_k(\Delta t_k), \quad v_R = \text{median}_k(\Delta \omega_k). \quad (22)$$

The next temporal proposal is

$$\hat{t}_{k+1} = t_k + v_t, \quad (23)$$

$$\hat{R}_{k+1} = \text{Exp}([v_R]_{\times}) R_k. \quad (24)$$

The median is used for robustness. Consider one-dimensional translation increments

$$[0.10, 0.11, 0.09, 2.00] \text{ m/frame}. \quad (25)$$

The mean is 0.575 m/frame, while the median is 0.105 m/frame. A single bad pose therefore corrupts the mean much more strongly. The same issue occurs when a temporary front/rear flip creates a rotation increment near 180° .

The median is less statistically efficient than the mean when all errors are small and Gaussian, and it can be less smooth. The behavior is configurable:

```

"temporal": {
  "window_size": 5,
  "robust_velocity": true
}

```

Setting `robust_velocity` to `false` replaces the median with the mean.

5 Optical-flow proposal

Features inside the previous object mask are tracked with pyramidal Lucas–Kanade optical flow. RANSAC fits a 2D partial affine/similarity model

$$u_t \approx Au_{t-1}, \quad (26)$$

which contains image translation, in-plane rotation θ , and scale s . The transformed object center supplies the pixel displacement. Image scale is converted approximately into a depth update:

$$\Delta \log z = -\log s, \quad z_{\text{new}} = z_{\text{old}} e^{-\log s} = \frac{z_{\text{old}}}{s}. \quad (27)$$

Thus, when the object becomes larger in the image ($s > 1$), the proposed depth decreases. The in-plane rotation update is

$$R_{\text{new}} = R_z(\theta)R_{\text{old}}. \quad (28)$$

Flow confidence is

$$c_{\text{flow}} = \text{clip} \left(r_{\text{inlier}} \min \left(\frac{N_{\text{inlier}}}{30}, 1 \right), 0, 1 \right). \quad (29)$$

The implementation is in `tracking/flow.py`.

6 Adaptive pose-hypothesis generation

Depending on the tracking state, the candidate set may combine:

- temporal propagation;
- optical-flow propagation;
- the previous pose;
- GigaPose top- K predictions;
- optional auxiliary predictions;
- 180° flipped poses;
- image-center, log-depth, translation, and rotation offsets.

For a pose with translation t , its projected center is

$$u = \pi(Kt). \quad (30)$$

Given a pixel offset Δu and log-depth offset $\Delta \log z$, define

$$z' = ze^{\Delta \log z} \quad (31)$$

and

$$t' = z' \frac{K^{-1} \begin{bmatrix} u + \Delta u \\ 1 \end{bmatrix}}{\left(K^{-1} \begin{bmatrix} u + \Delta u \\ 1 \end{bmatrix} \right)_z}. \quad (32)$$

Equivalent candidates are removed when both

$$\|t_i - t_j\|_2 \leq \tau_t \quad (33)$$

and

$$\|\text{Log}(R_i R_j^T)\|_2 \leq \tau_R. \quad (34)$$

Normal tracks follow a cheap local path. Uncertain and lost tracks receive a larger candidate set, and periodic global GigaPose hypotheses provide a way to escape accumulated drift. Candidate generation is implemented in `tracking/hypotheses.py`.

7 CAD alignment evidence

For each pose hypothesis, the CAD renderer produces a rendered mask M_r and rendered depth D_r . These are compared with the observed detection mask M_o , observed box B_o , and observed depth D_o .

7.1 Silhouette and bounding-box errors

$$E_{\text{sil}} = 1 - \frac{|M_r \cap M_o|}{|M_r \cup M_o|}, \quad (35)$$

$$E_{\text{bbox}} = 1 - \text{IoU}(B_r, B_o). \quad (36)$$

7.2 Symmetric edge error

Let ∂M denote a mask boundary and let $d(x, S)$ be the distance from point x to set S . The symmetric edge error is

$$E_{\text{edge}} = \text{clip} \left(\frac{\frac{1}{2} \left[\frac{1}{|\partial M_r|} \sum_{p \in \partial M_r} d(p, \partial M_o) + \frac{1}{|\partial M_o|} \sum_{q \in \partial M_o} d(q, \partial M_r) \right]}{d_{\text{max}}}, 0, 1 \right). \quad (37)$$

This is a normalized, truncated, symmetric Chamfer-style distance.

7.3 Depth error

Depth is evaluated only on pixels that are valid in both masks and depth maps:

$$\Omega = M_r \cap M_o \cap \{D_r > 0\} \cap \{D_o > 0\}. \quad (38)$$

The robust log-depth error is

$$E_{\text{depth}} = \text{clip} \left(\frac{\text{median}_{p \in \Omega} |\log(D_r(p) + \epsilon) - \log(D_o(p) + \epsilon)|}{\log(1 + d_{\text{trunc}})}, 0, 1 \right). \quad (39)$$

If depth is unavailable, the depth term and its weight are removed. Missing depth is not treated as a zero-error match.

7.4 Measurement and motion errors

The term called `feature_error` in the implementation currently uses the incoming GigaPose or auxiliary-prediction score:

$$E_{\text{feature}} = 1 - \text{clip}(s_{\text{measurement}}, 0, 1). \quad (40)$$

It is not a new DINO feature comparison. Relative to a temporal reference pose, the motion error is

$$E_{\text{motion}} = \frac{1}{2} \left[\min \left(\frac{\|t - t_{\text{motion}}\|_2}{s_t}, 1 \right) + \min \left(\frac{\theta(RR_{\text{motion}}^T)}{s_R}, 1 \right) \right], \quad (41)$$

with default scales

$$s_t = 1.5 \text{ m}, \quad s_R = 35^\circ. \quad (42)$$

7.5 Combined candidate score

The image-evidence error is

$$E_{\text{evidence}} = w_f E_{\text{feature}} + w_s E_{\text{sil}} + w_e E_{\text{edge}} + w_d E_{\text{depth}} + w_b E_{\text{bbox}}. \quad (43)$$

The full error is

$$E_{\text{total}} = E_{\text{evidence}} + w_m E_{\text{motion}}. \quad (44)$$

Default weights are

$$(w_f, w_s, w_e, w_d, w_b, w_m) = (0.15, 2.0, 0.75, 0.5, 0.75, 0.12). \quad (45)$$

Candidates are ranked by increasing E_{total} . Visual confidence deliberately excludes the motion prior:

$$c_{\text{visual}} = \text{clip} \left(1 - \frac{E_{\text{evidence}}}{w_f + w_s + w_e + w_d + w_b}, 0, 1 \right), \quad (46)$$

where unavailable terms and weights are omitted. Excluding motion prevents a temporally self-consistent but visually wrong track from becoming permanently overconfident. Scoring is implemented in `tracking/scoring.py`.

8 Derivative-free local refinement

The refiner performs coordinate search around each beam candidate. It tests

$$u_x \pm \Delta u, \quad u_y \pm \Delta u, \quad \log z \pm \Delta \log z, \quad (47)$$

and, by default, in-plane rotations

$$R_z(\pm\Delta\theta)R. \quad (48)$$

Full rotation refinement can additionally search the x and y axes. At iteration k , the search step decays according to

$$\Delta_{k+1} = \gamma \Delta_k, \quad \gamma = 0.55 \text{ by default.} \quad (49)$$

Only the lowest-error beam candidates survive. This resembles coordinate descent with beam search, but it optimizes a CAD-rendering objective without back-propagating through the renderer. See `tracking/refinement.py`.

9 Confidence, state transitions, and global recovery

If the two best candidates have $E_1 \leq E_2$, their separation reliability is

$$r = \text{clip} \left(\frac{E_2 - E_1}{m_{\min}}, 0, 1 \right). \quad (50)$$

Final confidence is

$$c = c_{\text{visual}}(0.75 + 0.25r). \quad (51)$$

An ambiguous result can therefore lose up to 25 percent of its visual confidence. Default state transitions are

$$\begin{cases} \text{normal,} & c \geq 0.67, \\ \text{uncertain,} & 0.35 \leq c < 0.67, \\ \text{lost,} & c < 0.35. \end{cases} \quad (52)$$

Uncertain and lost states perform broader searches. Periodic global hypotheses prevent unbounded drift. Optional same-frame recovery escalates from the cheap normal search to uncertain/lost search before the current frame is finalized. State management is implemented in `tracking/tracker.py`.

Part II

Learned recovery head

10 Recovery-data generation

Recovery training begins with a ground-truth pose

$$T^* = [R^*, t^*]. \quad (53)$$

The data generator creates synthetic candidate poses $\tilde{T} = [\tilde{R}, \tilde{t}]$, including clean poses, 180° flips, moderate and large image-center errors, log-depth errors, full 3D rotation errors, and partial synthetic occlusions.

Each candidate is rendered and summarized by the 11-dimensional vector

$$x = [E_f, E_{\text{sil}}, E_{\text{edge}}, E_{\text{depth}}, E_{\text{bbox}}, E_{\text{motion}}, \Delta u_x, \Delta u_y, \log(A_r/A_o), \log(D_r/D_o), s_{\text{measurement}}]. \quad (54)$$

Candidates generated from the same physical object instance share a group ID. Train/validation splitting is performed by group so perturbations of the same instance cannot leak across the split. Generation is implemented in `tracking/generate_recovery_dataset.py`.

11 Recovery targets

The desired left-applied rotation correction is

$$\omega^* = \text{Log} \left(R^* \tilde{R}^T \right), \quad (55)$$

which satisfies

$$R^* = \text{Exp}([\omega^*]_{\times}) \tilde{R}. \quad (56)$$

The desired camera-frame translation correction is

$$\Delta t^* = t^* - \tilde{t}. \quad (57)$$

The binary acceptable-pose target is

$$y = \mathbf{1} [\|t^* - \tilde{t}\|_2 \leq 0.15 \text{ m} \wedge E_R \leq 10^\circ]. \quad (58)$$

The scalar quality target is

$$q^* = \frac{\|t^* - \tilde{t}\|_2}{0.15 \text{ m}} + \frac{E_R}{10^\circ}. \quad (59)$$

Lower q^* indicates a better candidate.

12 Recovery network

Input features are standardized using statistics computed only from the training groups:

$$\bar{x} = \frac{x - \mu_x}{\sigma_x}. \quad (60)$$

With the default hidden dimension, the multilayer perceptron is

$$11 \longrightarrow 128 \longrightarrow 128 \longrightarrow 8. \quad (61)$$

It predicts

$$(\hat{\omega}_{\text{raw}}, \Delta \hat{t}_{\text{raw}}, \hat{a}, \hat{q}), \quad (62)$$

comprising three raw rotation values, three raw translation values, a confidence logit, and a scalar log-quality estimate.

An unconstrained vector is mapped to an open ball of radius r_{max} by

$$B(v; r_{\text{max}}) = r_{\text{max}} \tanh(\|v\|_2) \frac{v}{\max(\|v\|_2, \epsilon)}. \quad (63)$$

Hence

$$\hat{\omega} = B(\hat{\omega}_{\text{raw}}; \omega_{\text{max}}), \quad (64)$$

$$\Delta \hat{t} = B(\Delta \hat{t}_{\text{raw}}; t_{\text{max}}). \quad (65)$$

This prevents the learned head from applying arbitrarily large corrections. The network is defined in `tracking/recovery.py`.

13 Recovery training objective

For scalar residual r , PyTorch’s default Smooth-L1 penalty is

$$\rho(r) = \begin{cases} \frac{1}{2}r^2, & |r| < 1, \\ |r| - \frac{1}{2}, & |r| \geq 1. \end{cases} \quad (66)$$

The normalized rotation and translation losses are

$$L_R = \text{SmoothL1} \left(\frac{\hat{\omega}}{\omega_{\text{max}}}, \frac{\omega^*}{\omega_{\text{max}}} \right), \quad (67)$$

$$L_t = \text{SmoothL1} \left(\frac{\Delta \hat{t}}{t_{\text{max}}}, \frac{\Delta t^*}{t_{\text{max}}} \right). \quad (68)$$

The binary confidence loss is

$$L_c = -y \log \sigma(\hat{a}) - (1 - y) \log (1 - \sigma(\hat{a})). \quad (69)$$

The quality loss predicts a compressed log-quality target:

$$L_q = \text{SmoothL1}(\hat{q}, \log(1 + q^*)). \quad (70)$$

For candidates i and j from the same object instance, if $q_i^* < q_j^*$, candidate i should have lower predicted quality. The pairwise ranking penalty is

$$L_{\text{rank}}^{ij} = \max(0, m + \hat{q}_i - \hat{q}_j), \quad m = 0.1 \text{ by default.} \quad (71)$$

The total objective is

$$L = \lambda_R L_R + \lambda_t L_t + \lambda_c L_c + \lambda_q L_q + \lambda_{\text{rank}} L_{\text{rank}}. \quad (72)$$

Default weights are

$$(\lambda_R, \lambda_t, \lambda_c, \lambda_q, \lambda_{\text{rank}}) = (1, 1, 0.25, 0.20, 0.20). \quad (73)$$

The head is optimized using AdamW with default weight decay 10^{-4} . Gradient norm is clipped to 5. Validation uses the same total objective and early stopping monitors validation L . Training and validation loss components can optionally be logged to Weights & Biases. See `tracking/train_recovery.py`.

14 Using the learned correction at runtime

The learned head runs only when a track is uncertain or lost and a recovery checkpoint is supplied. For a candidate pose,

$$R_{\text{corrected}} = \text{Exp}([\hat{\omega}]_{\times}) R_{\text{candidate}}, \quad (74)$$

$$t_{\text{corrected}} = t_{\text{candidate}} + \Delta \hat{t}. \quad (75)$$

The predicted quality is converted to a score

$$s_q = e^{-\max(\hat{q}, 0)}, \quad (76)$$

and the proposal's measurement score becomes

$$s_{\text{new}} = \frac{s_{\text{old}} + \sigma(\hat{a}) + s_q}{3}. \quad (77)$$

The corrected proposal is not accepted blindly. It returns to the CAD refiner and is compared with all other candidates using silhouette, edge, depth, box, measurement, and motion evidence. The complete self-correcting loop is

$$\begin{aligned} \text{temporal/global candidates} &\rightarrow \text{CAD scoring} \rightarrow \text{learned correction} \\ &\rightarrow \text{CAD refinement} \rightarrow \text{verified pose}. \end{aligned}$$

The approach can improve a bad initial GigaPose pose when candidate generation or the recovery head reaches the correct alignment basin. It cannot guarantee recovery when all candidates are far from the true pose or when visual evidence is fundamentally ambiguous.

15 Relevant implementation files

- Pose geometry: `tracking/geometry.py`
- Multi-car association: `tracking/association.py`
- Optical flow: `tracking/flow.py`
- Candidate generation: `tracking/hypotheses.py`
- CAD scoring: `tracking/scoring.py`
- Local refinement: `tracking/refinement.py`

- State machine: `tracking/tracker.py`
- Recovery network: `tracking/recovery.py`
- Recovery-data generation: `tracking/generate_recovery_dataset.py`
- Recovery training: `tracking/train_recovery.py`
- Command-line runtime: `tracking/run_tracking.py`